

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO BÀI TẬP LỚN
MÔN: TRÍ TUỆ NHÂN TẠO

TRÒ CHƠI CỜ CARO AI

Sử dụng thuật toán Minimax và Alpha-Beta Pruning

Sinh viên 1:	Nguyễn Chí Phong	24020268
Sinh viên 2:	Vũ Văn Mạnh	23020109
Sinh viên 3:	Đỗ Thành An	23021204
Lớp:	INT3401 4	
Giảng viên hướng dẫn:	PGS.TS. Nguyễn Phương Thái	

Hà Nội, năm 2026

Mục lục

1	Giới thiệu bài toán	2
1.1	Mô tả bài toán cờ Caro	2
1.2	Luật chơi và điều kiện thắng	2
2	Cài đặt trạng thái bàn cờ	2
2.1	Cách biểu diễn trạng thái bàn cờ	2
2.2	Cách sinh nước đi hợp lệ	3
2.3	Cách kiểm tra trạng thái kết thúc	3
3	Thuật toán Minimax	4
3.1	Nguyên lý hoạt động	4
3.2	Cài đặt	4
3.3	Độ phức tạp	5
4	Thuật toán Alpha-Beta Pruning	5
4.1	Nguyên lý hoạt động	5
4.2	Cài đặt	5
4.3	So sánh với Minimax	6
5	Hàm đánh giá trạng thái	6
5.1	Nguyên lý	7
5.2	Bảng điểm số các mẫu	7
5.3	Cài đặt	7
6	Thiết kế các trạng thái thử nghiệm	9
7	Bảng kết quả thực nghiệm	9
8	Nhận xét và đánh giá	10
8.1	Nhận xét về số trạng thái đã xét và thời gian chạy	10
8.2	Nhận xét về ảnh hưởng của độ sâu tìm kiếm	11
8.3	Ưu điểm và hạn chế của chương trình	11
9	Kết luận và hướng phát triển	12
9.1	Kết luận	12
9.2	Hướng phát triển	12
10	Link Repository	12
11	Tài liệu tham khảo	13

1 Giới thiệu bài toán

1.1 Mô tả bài toán cờ Caro

Cờ Caro (hay Gomoku) là trò chơi chiến thuật giữa hai người chơi trên bàn cờ hình vuông. Hai người chơi lần lượt đặt quân cờ (ký hiệu X và O) vào các ô trống trên bàn cờ. Mục tiêu là tạo được một đường thẳng gồm 4 quân cờ liên tiếp theo hàng ngang, hàng dọc hoặc đường chéo.

Trong bài tập lớn này, chúng tôi cài đặt trò chơi cờ Caro với giao diện đồ họa sử dụng thư viện Pygame, trong đó người chơi đấu với máy tính. Máy tính sử dụng các thuật toán tìm kiếm có giới hạn độ sâu (Minimax và Alpha-Beta Pruning) để chọn nước đi tối ưu.

1.2 Luật chơi và điều kiện thắng

- Bàn cờ có kích thước tối thiểu 9×9 , có thể chọn 9×9 , 12×12 hoặc 15×15 .
- Hai người chơi lần lượt đánh quân vào các ô trống.
- Người chơi ký hiệu là **X**, máy ký hiệu là **O**, ô trống là dấu chấm (.).
- **Điều kiện thắng:** Người chơi nào có **4 quân liên tiếp** theo hàng ngang, hàng dọc hoặc đường chéo sẽ thắng.
- Không xét luật chặn hai đầu.
- Nếu bàn cờ đầy và không có người thắng thì kết quả là **hòa**.

2 Cài đặt trạng thái bàn cờ

2.1 Cách biểu diễn trạng thái bàn cờ

Bàn cờ được biểu diễn bằng một ma trận 2 chiều kích thước $n \times n$, trong đó mỗi ô có thể nhận một trong ba giá trị:

- '.' (EMPTY): ô trống
- 'X' (PLAYER_X): quân của người chơi
- 'O' (PLAYER_O): quân của máy

Lớp Board trong file board.py quản lý trạng thái bàn cờ:

```
1 class Board:
2     def __init__(self, size=15):
3         self.size = size
4         self.grid = [[EMPTY] * size for _ in range(size)]
5         self.move_history = []
6         self.move_count = 0
```

Ngoài ra, lớp Board còn lưu trữ:

- move_history: lịch sử các nước đi (dùng cho undo move trong Minimax)
- move_count: số quân đã đánh (để kiểm tra bàn cờ đầy)

2.2 Cách sinh nước đi hợp lệ

Chương trình cung cấp hai phương thức sinh nước đi:

1. **Sinh tất cả nước đi hợp lệ** (`get_valid_moves`): Duyệt toàn bộ bàn cờ, trả về danh sách các ô trống. Độ phức tạp $O(n^2)$.

2. **Sinh nước đi gần quân đã đánh** (`get_nearby_moves`): Chỉ xét các ô trống nằm trong bán kính r (mặc định $r = 2$) quanh các quân đã đánh. Phương thức này giúp giảm đáng kể số nhánh cần duyệt trong thuật toán tìm kiếm.

```

1 def get_nearby_moves(self, radius=2):
2     if self.move_count == 0:
3         center = self.size // 2
4         return [(center, center)]
5
6     occupied = set()
7     for r in range(self.size):
8         for c in range(self.size):
9             if self.grid[r][c] != EMPTY:
10                for dr in range(-radius, radius + 1):
11                    for dc in range(-radius, radius + 1):
12                        nr, nc = r + dr, c + dc
13                        if 0 <= nr < self.size and 0 <= nc < self.size:
14                            if self.grid[nr][nc] == EMPTY:
15                                occupied.add((nr, nc))
16     return sorted(occupied) if occupied else self.get_valid_moves()

```

2.3 Cách kiểm tra trạng thái kết thúc

Trạng thái kết thúc được kiểm tra sau mỗi nước đi bằng hàm `check_winner(row, col)`:

1. Từ vị trí vừa đánh (`row, col`), duyệt 4 hướng: ngang, dọc, chéo xuống, chéo lên.
2. Với mỗi hướng, đếm số quân liên tiếp cùng loại theo cả 2 chiều.
3. Nếu tổng số quân liên tiếp ≥ 4 (`WIN_LENGTH`), trả về người thắng và danh sách các ô thắng.
4. Nếu không ai thắng và bàn cờ đầy (`is_full()`), kết quả là hòa.

```

1 def check_winner(self, row, col):
2     player = self.grid[row][col]
3     directions = [(0, 1), (1, 0), (1, 1), (1, -1)]
4
5     for dr, dc in directions:
6         count = 1
7         win_cells = [(row, col)]
8         # Count positive direction
9         r, c = row + dr, col + dc
10        while (0 <= r < self.size and 0 <= c < self.size

```

```

11         and self.grid[r][c] == player):
12             count += 1
13             win_cells.append((r, c))
14             r += dr; c += dc
15         # Count negative direction
16         r, c = row - dr, col - dc
17         while (0 <= r < self.size and 0 <= c < self.size
18               and self.grid[r][c] == player):
19             count += 1
20             win_cells.append((r, c))
21             r -= dr; c -= dc
22
23         if count >= WIN_LENGTH:
24             return player, win_cells
25     return None

```

3 Thuật toán Minimax

3.1 Nguyên lý hoạt động

Minimax là thuật toán tìm kiếm cây trò chơi (game tree) hoạt động dựa trên nguyên lý:

- Người chơi MAX (máy): Chọn nước đi có giá trị **lớn nhất**.
- Người chơi MIN (đối thủ): Chọn nước đi có giá trị **nhỏ nhất**.

Thuật toán duyệt tất cả các trường hợp có thể xảy ra đến một độ sâu giới hạn, sau đó sử dụng hàm đánh giá để ước lượng giá trị của các trạng thái lá.

3.2 Cài đặt

```

1 def _minimax(self, board, depth, maximizing):
2     self.last_states += 1
3
4     # Check terminal state
5     last = board.move_history[-1] if board.move_history else None
6     if last:
7         result = board.check_winner(last[0], last[1])
8         if result:
9             return 10000000 if result[0] == self.player else
              -10000000
10
11     if board.is_full():
12         return 0
13
14     # Depth limit reached -> use evaluation function
15     if depth == 0:
16         return evaluate(board, self.player, self.opponent)
17

```

```

18     moves = board.get_nearby_moves(radius=2)
19
20     if maximizing:
21         max_eval = float('-inf')
22         for r, c in moves:
23             board.make_move(r, c, self.player)
24             score = self._minimax(board, depth - 1, False)
25             board.undo_move(r, c)
26             max_eval = max(max_eval, score)
27         return max_eval
28     else:
29         min_eval = float('inf')
30         for r, c in moves:
31             board.make_move(r, c, self.opponent)
32             score = self._minimax(board, depth - 1, True)
33             board.undo_move(r, c)
34             min_eval = min(min_eval, score)
35         return min_eval

```

3.3 Độ phức tạp

Độ phức tạp của Minimax là $O(b^d)$, trong đó:

- b : branching factor (số nước đi hợp lệ trung bình tại mỗi nút)
- d : độ sâu tìm kiếm

Với cờ Caro trên bàn cờ 15×15 và sử dụng `get_nearby_moves(radius=2)`, giá trị b giảm đáng kể so với duyệt toàn bộ bàn cờ.

4 Thuật toán Alpha-Beta Pruning

4.1 Nguyên lý hoạt động

Alpha-Beta Pruning là cải tiến của Minimax, giúp **cắt bỏ các nhánh không cần thiết** mà vẫn cho kết quả tương đương. Hai biến được sử dụng:

- **Alpha** (α): Giá trị tốt nhất hiện tại mà người chơi MAX có thể đảm bảo.
- **Beta** (β): Giá trị tốt nhất hiện tại mà người chơi MIN có thể đảm bảo.

Điều kiện cắt nhánh: Khi $\beta \leq \alpha$, nhánh hiện tại không cần duyệt thêm vì không thể cho kết quả tốt hơn.

4.2 Cài đặt

```

1 def _alpha_beta(self, board, depth, alpha, beta, maximizing):
2     self.last_states += 1
3
4     # Check terminal state (same as Minimax)
5
6     if depth == 0:
7         return evaluate(board, self.player, self.opponent)
8
9     moves = board.get_nearby_moves(radius=2)
10
11    if maximizing:
12        max_eval = float('-inf')
13        for r, c in moves:
14            board.make_move(r, c, self.player)
15            score = self._alpha_beta(board, depth-1, alpha, beta,
16                                    False)
17            board.undo_move(r, c)
18            max_eval = max(max_eval, score)
19            alpha = max(alpha, max_eval)
20            if beta <= alpha:
21                break # Prune
22        return max_eval
23    else:
24        min_eval = float('inf')
25        for r, c in moves:
26            board.make_move(r, c, self.opponent)
27            score = self._alpha_beta(board, depth-1, alpha, beta,
28                                    True)
29            board.undo_move(r, c)
30            min_eval = min(min_eval, score)
31            beta = min(beta, min_eval)
32            if beta <= alpha:
33                break # Prune
34        return min_eval

```

4.3 So sánh với Minimax

- Minimax duyệt **tất cả** các nhánh: $O(b^d)$
- Alpha-Beta cắt bỏ nhánh dư thừa: $O(b^{d/2})$ trong trường hợp tốt nhất
- Cả hai thuật toán cho **cùng kết quả** (cùng nước đi được chọn)
- Alpha-Beta giảm đáng kể số trạng thái đã xét và thời gian chạy

5 Hàm đánh giá trạng thái

Khi đạt đến giới hạn độ sâu, thuật toán sử dụng hàm đánh giá heuristic để ước lượng giá trị của trạng thái bàn cờ.

5.1 Nguyên lý

Hàm đánh giá xem xét các mẫu quân cờ (patterns) trên bàn cờ:

- Số quân liên tiếp (count): 1, 2, 3, 4+
- Số đầu mở (open_ends): 0, 1, 2

Điểm số được tính theo công thức:

$$\text{Score} = \text{Score}_{AI} - k \times \text{Score}_{Oith}$$

với $k = 1.2$ là hệ số ưu tiên phòng thủ.

5.2 Bảng điểm số các mẫu

Số quân	Đầu mở	Điểm số
≥ 4	-	10 000 000 (thắng)
3	2	5 000 000 (thắng chắc)
3	1	20 000
2	2	5 000
2	1	500
1	2	10

Bảng 1: Bảng điểm số cho các mẫu quân cờ

Nhận xét: Mẫu “3 quân, 2 đầu mở” được đánh giá rất cao (5 000 000) vì đây là thế thắng chắc – đối thủ chỉ chặn được một đầu, đầu còn lại sẽ hoàn thành 4 quân.

5.3 Cài đặt

```

1 def score_pattern(count, open_ends):
2     if count >= 4:
3         return 10_000_000
4     if count == 3:
5         if open_ends == 2:
6             return 5_000_000
7         elif open_ends == 1:
8             return 20_000
9     if count == 2:
10        if open_ends == 2:
11            return 5_000
12        elif open_ends == 1:
13            return 500
14    if count == 1:
15        if open_ends == 2:
16            return 10
17    return 0
18
19 def evaluate_player(board, player):

```



```

20     total_score = 0
21     directions = [(0,1), (1,0), (1,1), (1,-1)]
22     for row in range(board.size):
23         for col in range(board.size):
24             if board.grid[row][col] == player:
25                 for dr, dc in directions:
26                     # Skip if already counted from start of chain
27                     prev_row, prev_col = row - dr, col - dc
28                     if (0 <= prev_row < board.size
29                         and 0 <= prev_col < board.size
30                         and board.grid[prev_row][prev_col] ==
31                             player):
32                         continue
33                     count, open_ends = analyze_line(
34                         board, row, col, dr, dc, player
35                     )
36                     total_score += score_pattern(count, open_ends)
37
38 def evaluate(board, ai_player, opponent):
39     if check_win(board, ai_player):
40         return WIN_SCORE
41
42     if check_win(board, opponent):
43         return -WIN_SCORE
44
45     ai_score = evaluate_player(board, ai_player)
46     opponent_score = evaluate_player(board, opponent)
47
48     return ai_score - HUMAN_SCORE_MULTIPLIER * opponent_score
49
50 def analyze_line(board, row, col, dr, dc, player):
51     size = board.size
52     count = 0
53
54     r, c = row, col
55
56     while (
57         0 <= r < size
58         and 0 <= c < size
59         and board.grid[r][c] == player
60     ):
61         count += 1
62         r += dr
63         c += dc
64
65     open_ends = 0
66
67     if (0 <= r < size and 0 <= c < size and board.grid[r][c] ==
68         EMPTY):
69         open_ends += 1

```

```

69
70     before_row = row - dr
71     before_col = col - dc
72
73     if (
74         0 <= before_row < size
75         and 0 <= before_col < size
76         and board.grid[before_row][before_col] == EMPTY
77     ):
78         open_ends += 1
79
80     return count, open_ends

```

6 Thiết kế các trạng thái thử nghiệm

Để đánh giá hiệu quả của các thuật toán, chúng tôi thiết kế 6 trạng thái bàn cờ khác nhau:

STT	Trạng thái	Mô tả
1	Đầu ván	Bàn cờ trống hoặc chỉ có 1-2 quân
2	Giữa ván	Khoảng 10-15 quân đã đánh, thế cân bằng
3	AI thắng ngay	AI có 3 quân liên tiếp, chỉ cần 1 nước để thắng
4	Cần chặn	Người chơi có 3 quân liên tiếp, AI phải chặn
5	Cả hai tấn công	Cả hai bên đều có thể 3 quân
6	Nhiều nhánh	Nhiều nước đi hợp lệ, thuật toán phải duyệt nhiều nhánh

Bảng 2: Các trạng thái thử nghiệm

Mỗi trạng thái được chạy với cả hai thuật toán (Minimax và Alpha-Beta) với các độ sâu 1, 2, 3. Các chỉ số ghi nhận bao gồm: nước đi được chọn, giá trị đánh giá, số trạng thái đã xét và thời gian chạy.

7 Bảng kết quả thực nghiệm

Dưới đây là bảng tổng hợp kết quả chạy thực nghiệm so sánh hai thuật toán Minimax và Alpha-Beta trên 6 trạng thái bàn cờ với các độ sâu (Depth) từ 1 đến 3.

Để có cái nhìn tổng quan hơn về sự bùng nổ tổ hợp, bảng dưới đây thể hiện trung bình số trạng thái và thời gian chạy theo từng độ sâu:

Trạng thái	Depth	Move (Minimax)	Move ($\alpha - \beta$)	Trùng?	States (Minimax)	States ($\alpha - \beta$)	Giảm (%)	Thời gian $\alpha - \beta$ (ms)
opening	1	(4,3)	(4,3)	Có	32	32	0.00%	0.744
opening	2	(4,3)	(4,3)	Có	1132	451	60.16%	9.367
opening	3	(4,3)	(4,3)	Có	40722	8830	78.32%	208.561
midgame	1	(4,5)	(4,5)	Có	39	39	0.00%	1.202
midgame	2	(4,5)	(5,3)	Không	1548	693	55.23%	21.456
midgame	3	(4,2)	(4,2)	Có	59280	17631	70.26%	592.492
ai_win_now	1	(3,1)	(3,1)	Có	39	39	0.00%	1.135
ai_win_now	2	(3,1)	(3,1)	Có	1470	428	70.88%	12.759
ai_win_now	3	(3,1)	(3,1)	Có	56121	2710	95.17%	88.730
must_block	1	(1,3)	(1,3)	Có	35	35	0.00%	1.069
must_block	2	(3,5)	(3,5)	Có	1300	485	62.69%	13.500
must_block	3	(3,5)	(3,5)	Có	46702	10066	78.45%	282.740
both_attack	1	(5,4)	(5,4)	Có	38	38	0.00%	1.132
both_attack	2	(5,4)	(5,4)	Có	1402	443	68.40%	12.991
both_attack	3	(5,4)	(5,4)	Có	51380	5863	88.59%	193.193
many_branches	1	(4,3)	(4,3)	Có	40	40	0.00%	1.418
many_branches	2	(4,3)	(4,3)	Có	1600	422	73.62%	16.519
many_branches	3	(4,3)	(4,3)	Có	60880	9565	84.29%	389.316

Bảng 3: Chi tiết kết quả chạy thực nghiệm Minimax và Alpha-Beta

Depth	States (MM)	States ($\alpha - \beta$)	Giảm (%)	Time MM (ms)	Time $\alpha - \beta$ (ms)
1	37.2	37.2	0.00%	1.266	1.117
2	1408.7	487.0	65.17%	42.151	14.432
3	52514.2	9110.8	82.51%	1701.769	292.506

Bảng 4: Tổng hợp trung bình theo độ sâu (Depth)

8 Nhận xét và đánh giá

8.1 Nhận xét về số trạng thái đã xét và thời gian chạy

Dựa trên bảng số liệu thực nghiệm, chúng tôi rút ra các nhận xét sau về hiệu năng của thuật toán Alpha-Beta Pruning so với Minimax truyền thống:

- **Khả năng cắt tỉa xuất sắc:** Alpha-Beta làm giảm đáng kể số lượng trạng thái cần duyệt, đặc biệt là khi độ sâu tăng lên. Tính trung bình trong toàn bộ các lần chạy, Alpha-Beta giúp giảm khoảng 49.23% số trạng thái. Tuy nhiên, nếu chỉ xét riêng ở Depth 3, tỷ lệ cắt tỉa trung bình đạt tới 82.51%.
- **Trường hợp tối ưu nhất:** Khả năng cắt tỉa thể hiện sức mạnh vượt trội nhất ở kịch bản ai_win_now tại Depth 3. Thuật toán Minimax phải duyệt 56,121 trạng thái, trong khi Alpha-Beta nhờ phát hiện sớm được nhánh thắng đã cắt bỏ gần như toàn bộ cây, chỉ phải duyệt 2,710 trạng thái (giảm 95.17%). Thời gian tính toán nhờ đó cũng giảm từ 1765 ms xuống chỉ còn 88 ms.
- **Độ chính xác và tính nhất quán:** Dù bỏ qua một lượng lớn các nhánh tìm kiếm, Alpha-Beta vẫn đảm bảo tính chính xác tuyệt đối. Trong 18 lần chạy thử nghiệm, có 17/18 trường hợp Alpha-Beta chọn ra nước đi trùng khớp hoàn toàn với Minimax. Duy nhất 1 trường hợp khác biệt ở midgame Depth 2 (chọn (5,3) thay vì (4,5)), điều này xảy ra do hai nước đi có điểm heuristic đánh giá bằng nhau, thuật toán chỉ lấy nhánh được duyệt đến đầu tiên.

8.2 Nhận xét về ảnh hưởng của độ sâu tìm kiếm

Độ sâu (Depth) đóng vai trò quyết định đến cả thời gian tính toán và "độ thông minh" của AI. Qua thực nghiệm, chúng tôi nhận thấy:

- **Sự bùng nổ không gian trạng thái:** Khi tăng Depth từ 1 lên 3, số lượng trạng thái phải duyệt tăng theo cấp số nhân. Với Minimax, trung bình số trạng thái tăng từ 37.2 (Depth 1) lên 52, 514.2 (Depth 3). Dù sử dụng Alpha-Beta, con số này cũng tăng từ 37.2 lên 9, 110.8. Điều này minh chứng cho độ phức tạp $O(b^d)$ của bài toán cờ Caro.
- **Tầm nhìn chiến thuật được mở rộng:** Độ sâu lớn giúp AI khắc phục được "tầm nhìn hẹp" (horizon effect). Ví dụ điển hình ở kịch bản `must_block` (Người chơi sắp thắng):
 - Tại **Depth 1:** AI chỉ nhìn thấy lợi ích phòng thủ ngay trước mắt nên chọn một nước đi không liên quan là (1, 3), dẫn đến việc thua trận ngay ở lượt tiếp theo.
 - Tại **Depth 2 và 3:** AI có thể nhìn xa hơn 1-2 lượt của đối thủ, nhận ra mối đe dọa sinh tử và ngay lập tức thay đổi quyết định, đánh vào ô (3, 5) để chặn đầu chuỗi quân của người chơi.
- **Hiệu suất đạt mục tiêu:** Trong các kịch bản có nước đi mục tiêu rõ ràng (cần chặn hoặc cần thắng ngay), các thuật toán khi chạy ở độ sâu phù hợp đã chọn đúng nước đi tối ưu 8/9 lần. Thất bại duy nhất là trường hợp `must_block` tại Depth 1 như đã phân tích ở trên.

Kết luận lại, việc thiết lập Depth từ 3 đến 4 là mức cân bằng lý tưởng nhất cho trò chơi: Đủ sâu để AI hình thành các chiến thuật ép góc, chặn đối thủ, nhưng vẫn đảm bảo thời gian phản hồi ở mức chấp nhận được (dưới 1 giây cho mỗi nước đi với Alpha-Beta).

8.3 Ưu điểm và hạn chế của chương trình

Ưu điểm:

- Giao diện đồ họa trực quan, dễ sử dụng với Pygame.
- Hỗ trợ nhiều kích thước bàn cờ (9x9, 12x12, 15x15).
- Có thể chuyển đổi giữa Minimax và Alpha-Beta trong lúc chơi.
- Có thể thay đổi độ sâu tìm kiếm trong lúc chơi.
- Sử dụng `get_nearby_moves` để tối ưu hóa số nhánh cần duyệt.
- Hàm đánh giá phân biệt được "3 quân 2 đầu mở" là thế thắng chắc.

Hạn chế:

- Minimax chạy chậm ở độ sâu lớn (≥ 4).
- Hàm đánh giá heuristic chưa tối ưu hoàn toàn, có thể bỏ sót một số chiến thuật phức tạp.
- Chưa có cơ chế học tập hay cải tiến dần.
- Giao diện chưa hỗ trợ hoàn tác nước đi (undo).

9 Kết luận và hướng phát triển

9.1 Kết luận

Bài tập lớn đã cài đặt thành công trò chơi cờ Caro với AI sử dụng thuật toán Minimax và Alpha-Beta Pruning. Các thuật toán hoạt động đúng, có khả năng:

- Nhận diện và thực hiện nước thắng khi có cơ hội.
- Chặn đối thủ khi sắp thua.
- Ưu tiên các thế cờ có tiềm năng tấn công.

Alpha-Beta Pruning cho hiệu quả tốt hơn Minimax về mặt thời gian và số trạng thái đã xét, trong khi vẫn đảm bảo chất lượng nước đi tương đương.

9.2 Hướng phát triển

- Cải tiến hàm đánh giá: xem xét các mẫu phức tạp hơn (ví dụ: “2 quân + 1 quân” cách nhau 1 ô).
- Tối tiên thứ tự sinh nước đi: ưu tiên các ô có khả năng tạo chuỗi dài.
- Triển khai Iterative Deepening để cải thiện tìm kiếm.
- Thêm cơ chế lưu trữ kết quả đã tính (Transposition Table) để tránh tính toán lặp lại.
- Nâng cấp giao diện: undo, lưu/tải trận đấu, hiệu ứng âm thanh.

10 Link Repository

Source code của dự án được lưu trữ trên GitHub:

https://github.com/phongdepptrai/24020268_23020109_23021204_CaroAI

11 Tài liệu tham khảo

1. Russell, S. & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
2. Pygame Documentation. <https://www.pygame.org/docs/>
3. GeeksforGeeks. “Minimax Algorithm in Game Theory.” <https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory/>
4. GeeksforGeeks. “Alpha-Beta Pruning.” <https://www.geeksforgeeks.org/alpha-beta-pruning/>
5. Các repository mẫu tham khảo về cờ Caro AI trên GitHub.